

Day 3 - OOP Concepts

Game code is full of "things that are parts of other things", for example: an Orc is an Enemy, an Enemy is an Entity. Inheritance and polymorphism let you write code that works on the base type and does the right thing for each derived type. Done badly it becomes a tangle but done well it's how engines stay extensible. Today you build a small, correct class hierarchy.

Learning objectives

By the end of today you can:

- [ ] Design a class with the right access specifiers, keeping state `private` so the class alone enforces its invariants (encapsulation).
- [ ] Write constructors (incl. initializer lists) and a correct destructor, and predict construction/destruction order across a base, its derived type, and an owned member.
- [ ] Model relationships with inheritance ("is-a") and composition ("has-a"), and pick the right one without leaking internals.
- [ ] Achieve runtime polymorphism through a base pointer/reference: virtual functions marked `override`, a virtual destructor, and specializing behavior by delegating to the base version.
- [ ] Define an abstract class with a pure virtual function and explain why it can't be instantiated.
- [ ] Group related types under a namespace and read a qualified name (`std::vector`, `game::combat::Enemy`) as "which library or subsystem does this belong to".

Concepts

Classes & access specifiers

A class bundles state with the operations that keep it valid. `public` is the API callers use, `private` is state only the class touches, `protected` is state derived classes may touch. Keep data `private` so the class alone enforces its invariants.

```
#include <algorithm>   // std::max

class Health {
public:
    explicit Health(int max) : m_max(max), m_current(max) {}
    void damage(int amount){ m_current = std::max(0, m_current - amount);
}
    int  current() const{ return m_current; }   // invariant: 0 <= current
<= max
```

```
private:
    int m_max;
    int m_current;
};
```

❌ **Common Mistake:** exposing data members `public` and re-checking invariants at every call site. One call site forgets, and you get a corrupt object with nothing to point at.

✅ **Good Practice:** keep state `private` and enforce the invariant once, inside the class's own methods.

Constructors & destructors

Prefer a **member initializer list** over assignment in the body: members are initialised in *declaration order*, before the body runs. The destructor releases whatever the object owns. **Rule of zero/three/five:** if your members clean up after themselves (e.g. smart pointers), write no special members (rule of zero); if you manually manage a resource and thus need a destructor, you almost certainly also need the copy *and* move operations (rule of three/five).

```
class Enemy {
public:
    Enemy(std::string name, int hp)
        : m_name(std::move(name)), m_hp(hp) {}   // init list, in
declaration order
    ~Enemy() { /* release anything owned here */ }
private:
    std::string m_name;   // initialised first ...
    int         m_hp;     //   ... then this
};
```

❌ **Common Mistake:** listing initializers in a different order than the members are declared. The compiler follows declaration order, not your list, so a member can be initialised from one that's still uninitialised (and `-Wall -Werror` stops the build).

✅ **Good Practice:** keep the initializer list in declaration order, and let the warning be the check.

Inheritance

Inheritance models "is-a" (an `Enemy` *is an* `Entity`). You may instead use ("has-a") model when the relationship is really ownership of a part like `Enemy` owning a `Weapon`. Base sub objects are constructed before derived, and destroyed in reverse. `protected` members are visible to derived classes but not to outside code.

Keep hierarchies **shallow: two levels is the practical limit** (`Entity` → `Enemy`, and stop). At three levels you can't answer "what does this object actually do?" without opening three files, and a one-line change to the base ripples into types nobody remembers writing. Need a third variation? Make it a member (`Enemy` holds an `AIBehavior`) rather than a grandchild. **Multiple inheritance, deriving from two concrete bases, is not recommended at all**: names collide, a shared base gets duplicated into two sub-objects (the "diamond problem"), and construction order stops being obvious. C++ offers `virtual` inheritance to patch that, and needing a patch is the warning. If you want behaviour from two places, inherit the one that is genuinely "is-a" and compose the other as a member.

```
class Entity {
public:
    explicit Entity(std::string name) : m_name(std::move(name)) {}
protected:
    std::string m_name;   // derived classes can see this
};

class Enemy : public Entity {
public:
    Enemy(std::string name, int hp) : Entity(std::move(name)), m_hp(hp) {}
private:
    int m_hp;
};
```

❌ **Common Mistake:** reaching for inheritance to reuse code when the types aren't genuinely substitutable. The base ends up with methods that make no sense on half its children.

✅ **Good Practice:** say the relationship out loud first. "is-a" inherits; "has-a" composes, so hold the reused type as a member instead.

Polymorphism & virtual functions

A `virtual` function called through a base pointer or reference dispatches to the *derived* override at runtime (the object carries a hidden pointer to its type's vtable). This lets one loop drive many concrete types. Mark overrides `override` so the compiler checks the signature matches.

```
class Entity {
public:
    virtual std::string describe() const= 0;   // see abstract classes
below
    virtual ~Entity() = default;
};
class Player : public Entity {
public:
    std::string describe() const override{ return "Player"; }
};
class Enemy : public Entity {
public:
    std::string describe() const override{ return "Enemy"; }
};
```

```
void report(const Entity& e){ std::cout << e.describe() << '\n'; }  //
right one each time
```

❌ **Common Mistake:** expecting virtual dispatch through a *value*. Copying a derived object into an `Entity` slices it down to the base, so the base version quietly runs.

✅ **Good Practice:** handle polymorphic objects through pointers or references (`Entity&`, `std::unique_ptr<Entity>`).

Virtual destructors

Deleting a derived object through a base pointer only runs the derived destructor if the base destructor is `virtual`. Without it, the derived part's cleanup is skipped, which leaks (and is undefined behaviour). Any class meant to be a polymorphic base needs `virtual ~Base()`.

```
Entity* e = new Enemy(...);
delete e;   // runs ~Enemy only if ~Entity is virtual; otherwise leaks
Enemy's parts
```

Abstract classes

A **pure virtual** function (`= 0`) has no (required) definition and makes the class *abstract*: it can't be instantiated, only derived from. Use it to define an interface: the contract every subtype must implement.

```
class Entity {
public:
    virtual std::string describe() const= 0;   // pure virtual -> abstract
    virtual ~Entity() = default;
};
// Entity e;            // error: abstract, cannot instantiate
// Player p;  Enemy n;  // ok: they implement describe()
```

❌ **Common Mistake:** forgetting the `virtual ~Entity()` on the abstract base. An interface is exactly the case where you delete through the base pointer, so the derived part's cleanup never runs.

✅ **Good Practice:** give every polymorphic base a `virtual ~Base() = default;`.

Namespaces

A namespace is a surname for your types: it answers "who does this name belong to?". That's why every standard-library type is written `std::vector`, `std::string`, `std::cout`. The `std::` isn't noise, it's the type's home address, and it's what lets your own `sort` or `list` coexist with the library's. Game engines organise their code the same way, and not only to avoid collisions. Split the code into `game::combat`, `game::render`, and `game::net`, and a crash report that says the fault is in `game::combat::Enemy::takeDamage` tells you which subsystem and which team owns it before you open a single file. A bare `takeDamage` in a stack trace tells you nothing.

```cpp
namespace game::combat {          // C++17: nested namespace in one line
    class Enemy { /* ... */ };
}

game::combat::Enemy goblin("Goblin", 12);   // qualified: reads as
"combat's Enemy"

namespace gc = game::combat;                // alias, for a long name used
often
gc::Enemy orc("Orc", 40);
```

❌ **Common Mistake:** `using namespace std;` at the top of a header. It dumps every standard name into every file that includes it, so a later `std::count` or `std::data` collides with your own and the error points at a line you didn't write.

✅ **Good Practice:** write `std::` in full, and if a name is genuinely too long, alias it (`namespace gc = game::combat;`) inside the `.cpp` that needs it.

Exercises

The hierarchy you build in ex00 is reused and extended in ex01 and ex02. Target API:

```cpp
// Entity.hpp
class Entity {
public:
    explicit Entity(std::string name);
    virtual std::string describe() const= 0;   // pure virtual
    virtual ~Entity();                           // virtual!
protected:
    std::string m_name;
};

class Player : public Entity {
public:
    explicit Player(std::string name);          // name only: players have
no hp here
    std::string describe() const override;      // -> "Player <name>"
};

class Enemy : public Entity {
public:
    Enemy(std::string name, int hp);            // name AND hp, both
required
    std::string describe() const override;      // -> "Enemy <name> (hp N)"
private:
    int m_hp;
};
```

So `Player` is constructed from a name alone, while `Enemy` requires a name *and* an `hp` value. That extra state is exactly what its `describe()` prints.

ex00 - Entity hierarchy

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Implement an abstract `Entity` with concrete `Player` and `Enemy`, then drive a mixed set through one loop over a base-type container.*
📁	**Turn in directory**	`ex00/`
❗	**Turn in files**	**Entity.hpp/.cpp, Player.hpp/.cpp, Enemy.hpp/.cpp, main.cpp, Makefile**
🛠️	**Skills required**	*Abstract classes and pure virtual functions, `override`, virtual destructors, storing polymorphic objects behind a base pointer or smart pointer.*
🔍	**Search keywords**	*C++ abstract class pure virtual, override keyword, virtual destructor, base class pointer polymorphism, vector of unique_ptr*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<vector>`, `<memory>`, `<string>`

- **Goal:** Implement the abstract `Entity` above with concrete `Player` and `Enemy`. Correct access specifiers, `override` on each `describe()`, and a `virtual` destructor. In `main`, fill a `std::vector<Entity*>` (or `unique_ptr<Entity>`) with a mix and loop once, printing each `describe()`.
- **Done when:** The single loop prints the *right* description per element, and destroying every element leaks nothing.

Your `main` should read roughly like this: one container, one loop, no type-checking.

```
// main.cpp
std::vector<std::unique_ptr<Entity>> party;
party.push_back(std::make_unique<Player>("Aria"));
party.push_back(std::make_unique<Enemy>("Goblin", 12));
party.push_back(std::make_unique<Player>("Bran"));
party.push_back(std::make_unique<Enemy>("Orc", 40));

for (const auto& e : party)                 // single loop, base type only
    std::cout << e->describe() << '\n';
// unique_ptr frees every element here; with raw Entity* you'd delete in a
loop
```

Output example:

```
$ ./entities
Player Aria
Enemy Goblin (hp 12)
Player Bran
Enemy Orc (hp 40)
```

Each line comes from the *derived* `describe()` even though the loop only sees `Entity`. That's virtual dispatch doing its job.

ex01 - Construction & cleanup

.	.	.
💢	**Difficulty**	⭐⭐⭐⭐
🧠	**Objectives**	*Add a non-copyable `Inventory` that owns a heap array, give `Enemy` one as a member, and trace every constructor/destructor to see base-before-derived construction and its exact reverse on teardown.*
📁	**Turn in directory**	`ex01/`
❗	**Turn in files**	**reuse ex00's files plus Inventory.hpp/.cpp, main.cpp, Makefile**
🛠️	**Skills required**	*Member initializer lists, construction/destruction order across base, member, and derived, `new[]` / `delete[]` in constructor/destructor, deleting copy operations, composition (has-a).*
🔍	**Search keywords**	*C++ construction order base member derived, destruction reverse order, new[] delete[], deleted copy constructor, initializer_list, composition has-a*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<string>`, `<algorithm>`, `<initializer_list>`

- **Goal:** Add an `Inventory` class that owns a **heap-allocated** array of item-name strings: allocate it with `new[]` in the constructor, free it with `delete[]` in the destructor. Give every `Enemy` an `Inventory` member (an `Enemy` *has-a* bag). Then make **every** constructor and destructor (`Entity`, `Player`, `Enemy`, and `Inventory`) print one trace line: `+ <Class> <who>` when it is constructed, `<Class> <who>` when it is destroyed. In `main`, build one `Player` and one `Enemy` behind `Entity*`, print each `describe()`, then `delete` them through the base pointer.
- **Done when:** The trace shows **base-before-derived** on construction and the **exact reverse** on destruction (see the expected output below), and the leak detector reports zero leaks.

Extend the hierarchy with an owned resource. `Inventory` manages a raw buffer, so make it non-copyable: one owner, one `delete[]`.

```
// Inventory.hpp: a resource that must be released
class Inventory {
public:
    Inventory(const std::string& owner, std::initializer_list<std::string>
items);
    ~Inventory();                                       // delete[] here,
prints '-'

    Inventory(const Inventory&)            = delete;   // owns a raw
buffer:
    Inventory& operator=(const Inventory&) = delete;   //   forbid copies
(no double free)
private:
    std::string  m_owner;
    std::size_t  m_count;
    std::string* m_items;   // new std::string[m_count] of item names,
freed in ~Inventory
};

// Enemy.hpp: now owns a bag
class Enemy : public Entity {
public:
    Enemy(std::string name, int hp, std::initializer_list<std::string>
loot);
    ~Enemy();                           // implicitly virtual (base dtor is
virtual)
    std::string describe() const override;
private:
    int       m_hp;
    Inventory m_bag;   // built after the Entity base, destroyed before it
};
```

Drive it through the base type and then delete through the base pointer:

```
// main.cpp
Entity* hero    = new Player("Aria");
Entity* monster = new Enemy("Goblin", 12, {"dagger", "gold", "torch"});

std::cout << hero->describe()    << '\n';
std::cout << monster->describe() << '\n';

std::cout << "--- cleaning up ---\n";
delete hero;      // virtual ~Entity() dispatches to ~Player, then ~Entity
delete monster;   //   ... and to ~Enemy, then its Inventory, then ~Entity
```

Output example:

```
$ ./entities
+ Entity    Aria
+ Player    Aria
+ Entity    Goblin
+ Inventory Goblin (dagger, gold, torch)
+ Enemy     Goblin
```

```
Player Aria
Enemy Goblin (hp 12)
--- cleaning up ---
- Player    Aria
- Entity    Aria
- Enemy     Goblin
- Inventory Goblin (dagger, gold, torch)
- Entity    Goblin
```

Read the trace top to bottom: the `Enemy` builds `Entity` → `Inventory` member → `Enemy` body, then tears down in the exact reverse, `Enemy` body → `Inventory` → `Entity`. That reversal, driven through a base pointer, is the whole lesson.

ex02 - Damage & encapsulation

.	.	.
💢	**Difficulty**	⭐⭐⭐⭐
🧠	**Objectives**	*Keep health state private behind clamping `takeDamage` / `heal`, make `takeDamage` virtual, and have `Player` override it to soak damage before delegating to the base: encapsulation and polymorphism in one model.*
📁	**Turn in directory**	`ex02/`
❗	**Turn in files**	**ex00's files (extended), main.cpp, Makefile**
🛠️	**Skills required**	*Encapsulation and class invariants, virtual method overriding, delegating to a base implementation (`Base::method`), clamping with `std::min` / `std::max`.*
🔍	**Search keywords**	*C++ class invariant encapsulation, virtual function override, call base class method, std::min std::max clamp, private member access*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<string>`, `<algorithm>`

- **Goal:** Give `Entity` its own health state (`m_currentHp`, `m_maxHp`) kept **private**, guarding one invariant: `0 <= currentHp <= maxHp`. The only ways to change it are `takeDamage(int)` (clamps at 0) and `heal(int)` (clamps at `maxHp`). No setter exposes the raw value, so the class alone keeps itself valid.
  Make `takeDamage` **virtual**: `Enemy` takes the full hit (the base behaviour), while `Player` has armor and **overrides** `takeDamage` to soak part of the hit, then delegates to `Entity::takeDamage`. The derived version *augments* the base, it doesn't reimplement the clamping. Each `takeDamage` **logs** the hit (`<name> takes N damage (before -> after hp)`) so the fight is visible, and `describe()` reports current/max hp.

- **Done when:** Health never leaves `[0, maxHp]` (overkill clamps to 0, over-heal clamps to `maxHp`), and the *same* `takeDamage` call through an `Entity*` does full damage to an `Enemy` but reduced damage to a `Player`, matching the output below.

Health state stays private; exposing it publicly may lead to it being set to an invalid value otherwise. The only door to it is a pair of methods that keep it valid. Callers can never write an out-of-range value:

```
// Entity.hpp (additions)
class Entity {
public:
    Entity(std::string name, int maxHp);
    virtual void        takeDamage(int amount);   // default: full damage
    void                heal(int amount);
    bool                isAlive() const;          // currentHp > 0
    int                 currentHp() const;
    int                 maxHp() const;
    virtual std::string describe() const= 0;
    virtual ~Entity();
protected:
    std::string m_name;
private:
    int m_maxHp;
    int m_currentHp;   // invariant: 0 <= m_currentHp <= m_maxHp; only the
methods below touch it
};

// Entity.cpp: the invariant is enforced here, and every hit is logged
void Entity::takeDamage(int amount){
    int before  = m_currentHp;
    m_currentHp = std::max(0, m_currentHp - amount);          // clamp at 0
    std::cout << m_name << " takes " << amount << " damage ("
              << before << " -> " << m_currentHp << " hp)\n";
}
void Entity::heal(int amount){ m_currentHp = std::min(m_maxHp, m_currentHp
+ amount); }
```

`takeDamage` is where polymorphism does real work. The `Player` override reduces the hit, then calls the base version rather than re-clamping itself:

```
// Player.cpp: armor soaks part of the hit, min 1 damage gets through
void Player::takeDamage(int amount){
    int reduced = std::max(1, amount - m_armor);
    std::cout << m_name << "'s armor soaks " << (amount - reduced) <<
'\n';
    Entity::takeDamage(reduced);   // delegate: base applies the hit and
logs it
}
```

Deal the *same* hit through a base pointer and watch each type handle it its way:

```
// main.cpp
Enemy  goblin("Goblin", 12);      // 12 hp, no armor
Player hero("Aria", 30, 5);       // 30 hp, 5 armor
```

```
Entity* a = &goblin;              // same static type ...
Entity* b = &hero;               //   ... different runtime behavior
a->takeDamage(8);                // Enemy: full 8            -> 4/12
b->takeDamage(8);                // Player: 8 - 5 armor = 3  -> 27/30

std::cout << goblin.describe() << '\n';
std::cout << hero.describe()   << '\n';

goblin.takeDamage(100);          // overkill clamps at 0, never negative
hero.heal(999);                  // over-heal clamps at max, never above 30

std::cout << goblin.describe() << '\n';
std::cout << hero.describe()   << '\n';
```

Output example:

```
$ ./entities
Goblin takes 8 damage (12 -> 4 hp)
Aria's armor soaks 5
Aria takes 3 damage (30 -> 27 hp)
Enemy Goblin (hp 4/12)
Player Aria (hp 27/30, armor 5)
Goblin takes 100 damage (4 -> 0 hp)
Enemy Goblin (hp 0/12) [dead]
Player Aria (hp 30/30, armor 5)
```

The clamps prove the health state guards its own invariant no matter what callers throw at it. The two different log outputs from one `takeDamage(8)` call, a bare hit for the `Enemy` and an "armor soaks" line first for the `Player`, prove the override is chosen at runtime. That's encapsulation and polymorphism in one small model.

Constraints (every exercise)

- Compiles with `c++ -Wall -Wextra -Werror -std=c++17`.
- Zero leaks (carries over from Day 2).
- Base classes intended for inheritance have a `virtual` destructor.
- Overrides are marked `override`; data members stay `private` / `protected`.

How you're evaluated

An evaluation will be automatically scheduled with a peer and start at 6pm. Submit all the work you have done so far even if its not 100% complete.

Turning in your work

1. Create a new repository on GitHub named `gauntlet-day-03`, and keep it **public**. A private repo is one your evaluator cannot clone, and the evaluation cannot start without the code.
2. Commit today's work there, one directory per exercise (`ex00/`, `ex01/`, ...) exactly as each turn-in table says, `Makefile` included.
3. Push before the evaluation. Whatever sits on the default branch when the evaluation starts is what gets read.

4. When the evaluation starts, give your evaluator the link to the repository.

AI-written code is an immediate disqualification. Look things up and ask an AI what something does, that is a tool. Pasting in code you did not write is not.